

Sistemas Distribuídos — Aula 20

Algoritmos Distribuídos III: Replicação e Consistência de Dados · 22/10 · Prof. Leandro Borges

Por que replicar dados

Nas duas últimas aulas, olhamos para dois problemas centrais de coordenação em sistemas distribuídos: como um conjunto de processos chega a um acordo sobre um único valor mesmo com falhas e mensagens perdidas (consenso, Aula 18), e como capturar um retrato consistente do estado global de um sistema em execução, ou detectar coletivamente que um cálculo distribuído terminou (Aula 19). Em ambos os casos, havia implicitamente múltiplas cópias de informação circulando entre processos. Esta aula lida diretamente com o motivo pelo qual sistemas distribuídos mantêm múltiplas cópias dos mesmos dados — replicação — e com o problema que essa decisão cria: manter essas cópias de acordo entre si, ou seja, garantir algum grau de consistência.

Replicar dados — manter a mesma informação armazenada em mais de uma máquina, muitas vezes em data centers ou regiões geográficas diferentes — resolve três problemas práticos ao mesmo tempo. O primeiro é disponibilidade: se os dados existem em uma única máquina e ela falha, fica indisponível ou perde conectividade de rede, o serviço inteiro para. Com réplicas, o sistema continua respondendo a partir de qualquer cópia que ainda esteja acessível, e os dados não dependem de um único ponto de falha.

O segundo motivo é desempenho e latência. Manter cópias fisicamente mais próximas dos usuários reduz o tempo de resposta de cada requisição — um usuário no Brasil lendo de uma réplica em São Paulo, em vez de uma única cópia nos Estados Unidos, percebe uma aplicação sensivelmente mais rápida. Além disso, distribuir requisições de leitura entre várias réplicas evita que uma única máquina precise absorver toda a carga do sistema sozinha, o que aumenta a capacidade total de atendimento.

O terceiro motivo é tolerância a falhas, o fio condutor que já vem das Aulas 17 e 18: se uma réplica falha — por uma queda de energia, um disco corrompido ou uma partição de rede — o sistema continua operando a partir das réplicas restantes, e a cópia perdida pode ser reconstruída posteriormente a partir de uma réplica saudável. Nenhum desses três benefícios vem de graça: assim que existe mais de uma cópia do mesmo dado, surge a pergunta que ocupa o resto desta aula — o que acontece quando essas cópias, temporariamente, discordam entre si?

Modelos de consistência: forte e eventual

Um modelo de consistência é o contrato que o sistema oferece a quem lê e escreve dados: quais garantias existem sobre o que uma leitura pode retornar, dado o histórico de escritas já realizadas em qualquer réplica. No extremo mais rígido está a consistência forte, também chamada strict ou strong consistency: toda leitura, em qualquer réplica, retorna o valor da escrita mais recente, como se existisse uma única cópia lógica dos dados, mesmo que fisicamente existam várias. É o comportamento mais previsível e mais fácil de raciocinar para quem programa contra o sistema — mas tem um custo. Garantir que toda réplica reflita imediatamente a última escrita exige coordenação entre as réplicas a cada operação de escrita, o que aumenta a latência e, como veremos adiante no teorema CAP, reduz a disponibilidade do sistema exatamente quando ela seria mais necessária: durante uma falha de rede.

No outro extremo está a consistência eventual (eventual consistency), o modelo por trás de bancos de dados NoSQL amplamente usados em produção, como Amazon DynamoDB e Apache

Cassandra. Nesse modelo, escritas se propagam entre réplicas de forma assíncrona, em segundo plano; se não houver novas escritas por um tempo, todas as réplicas eventualmente convergem para o mesmo valor. A vantagem é dupla: baixa latência de escrita, porque o cliente não precisa esperar a propagação para todas as réplicas, e alta disponibilidade, porque uma réplica lenta ou temporariamente isolada não bloqueia o sistema inteiro. O custo é que, durante a janela de propagação, leituras em réplicas diferentes podem retornar valores diferentes — uma leitura pode devolver um dado desatualizado (stale), sem que isso seja tratado como erro pelo sistema.

A escolha entre esses dois extremos não é uma questão de qual é "melhor" em abstrato, mas de qual conjunto de garantias o domínio da aplicação exige. Um sistema bancário processando uma transferência não pode tolerar que duas réplicas discordem sobre o saldo de uma conta por nem um segundo — ali, o custo de coordenação da consistência forte é aceitável e necessário. Já um contador de curtidas em uma rede social, ou um cache de resultados de busca, tolera perfeitamente que réplicas divirjam por alguns segundos, e prefere de longe a baixa latência e a alta disponibilidade da consistência eventual.

Consistência causal e garantias de sessão: um meio-termo

Entre os dois extremos existem modelos intermediários, pensados para capturar exatamente as garantias que o usuário de uma aplicação realmente percebe, sem pagar o custo total da consistência forte. O mais estudado é a consistência causal: ela garante que operações causalmente relacionadas — por exemplo, uma escrita que depende do resultado de uma leitura anterior — sejam vistas por todos os processos na mesma ordem, preservando sempre a relação "causa antes do efeito". Um comentário em uma postagem, por exemplo, sempre aparece depois da postagem à qual ele responde, em qualquer réplica que o exiba, porque a escrita do comentário depende causalmente da leitura da postagem original. Já operações verdadeiramente concorrentes — sem nenhuma relação de causa e efeito entre si — podem ser vistas em ordens diferentes por réplicas diferentes, sem que isso viole nenhuma garantia. Sistemas como o COPS foram desenhados especificamente em torno dessa ideia.

Um conjunto relacionado de garantias, mais pontual e popular justamente em sistemas de consistência eventual, é chamado de garantias orientadas à sessão. Duas das mais comuns são read-your-writes (o próprio cliente sempre enxerga as escritas que ele mesmo acabou de fazer, mesmo que outros clientes ainda não as vejam) e monotonic reads (leituras sucessivas de um mesmo cliente nunca "andam para trás" no tempo, mesmo que ele seja atendido por réplicas diferentes em requisições diferentes). Essas garantias resolvem exatamente o tipo de inconsistência que mais incomoda um usuário final — como editar o próprio perfil e, ao recarregar a página, ver a versão antiga novamente — sem exigir a coordenação cara e global da consistência forte.

Estratégias de replicação: síncrona vs. assíncrona

Além do modelo de consistência que o sistema promete, existe uma decisão de engenharia sobre como, mecanicamente, a réplica é atualizada: replicação síncrona ou assíncrona. Na replicação síncrona, o coordenador só confirma a escrita ao cliente depois que a réplica, ou um quórum de réplicas, também confirmar que gravou o dado. Essa abordagem garante forte durabilidade e consistência — o dado nunca é reportado como salvo se não estiver de fato replicado — mas a latência da escrita fica limitada pela réplica mais lenta do grupo, e uma réplica indisponível pode travar novas escritas inteiramente, até que ela volte ou seja removida do quórum.

Na replicação assíncrona, o coordenador confirma a escrita ao cliente assim que grava localmente, e propaga a atualização às réplicas em segundo plano, sem que o cliente espere por essa propagação. Escritas são rápidas e o sistema tolera réplicas lentas ou temporariamente offline sem impacto perceptível para quem escreve. O custo é uma janela de réplicas

desatualizadas — e, mais grave, dados que já foram confirmados ao cliente podem se perder de fato se o nó principal falhar antes de conseguir propagar a atualização às demais réplicas. Não por acaso, replicação síncrona tende a andar de mãos dadas com consistência forte, e replicação assíncrona com consistência eventual — a escolha mecânica de como replicar e a garantia lógica que se oferece ao usuário são, na prática, a mesma decisão vista por dois ângulos.

Topologias de replicação: master-slave vs. multi-master

Uma segunda decisão de arquitetura, independente da anterior, é quem tem permissão para aceitar escritas. Na topologia master-slave, também chamada primary-backup, apenas o nó primário aceita escritas; os nós secundários geralmente só atendem leituras, recebendo as atualizações do primário por replicação. A grande vantagem é que conflitos de escrita simplesmente não ocorrem — existe sempre uma única fonte de verdade por vez, o que simplifica enormemente o raciocínio sobre o sistema. A desvantagem é que o primário se torna tanto um gargalo de escrita (toda escrita passa por ele) quanto um ponto único de falha até que um processo de failover — frequentemente apoiado em um algoritmo de eleição de líder, como os vistos na Aula 17, ou em consenso, como Raft, visto na Aula 18 — promova um novo primário.

Na topologia multi-master, qualquer réplica pode aceitar escritas diretamente, sem depender de um único nó coordenador. Isso elimina o gargalo de escrita e o ponto único de falha, melhorando a disponibilidade de escrita do sistema como um todo — mas abre a porta para escritas concorrentes na mesma chave, em réplicas diferentes, que precisam ser reconciliadas depois. Diferente do master-slave, onde a resolução de conflito é desnecessária, aqui ela é obrigatória, e existem estratégias específicas para isso: last-write-wins (a escrita com o timestamp mais recente vence, com o risco de descartar silenciosamente uma escrita legítima), vetores de versão (que detectam quando duas escritas são realmente conflitantes, em vez de uma simplesmente suceder a outra) e CRDTs — estruturas de dados replicadas sem conflito (Conflict-free Replicated Data Types), desenhadas matematicamente para que qualquer ordem de aplicação das escritas convirja para o mesmo resultado final, sem precisar de coordenação alguma.

O teorema CAP

Em 2000, Eric Brewer propôs, em uma palestra que ficou conhecida como a conjectura de Brewer, uma formulação que depois foi provada formalmente por Seth Gilbert e Nancy Lynch em 2002: o teorema CAP. O teorema afirma que um sistema distribuído que armazena dados replicados não pode garantir simultaneamente mais do que duas das três propriedades a seguir: Consistência (C) — toda leitura, em qualquer nó, retorna a escrita mais recente; Disponibilidade (A, de availability) — toda requisição recebe uma resposta, mesmo que não seja o dado mais recente; e Tolerância a Partição (P) — o sistema continua operando mesmo quando mensagens entre nós se perdem ou atrasam, ou seja, mesmo quando a rede se divide em grupos que não conseguem se comunicar entre si.

A intuição central por trás da prova é simples de enunciar: imagine dois nós que replicam o mesmo dado, e uma partição de rede impede que eles troquem mensagens entre si. Se um cliente escreve em um dos nós durante a partição, o outro nó não tem como saber dessa escrita — a mensagem que a propagaria não consegue atravessar a partição. Agora, se um segundo cliente faz uma leitura no outro nó, o sistema tem exatamente duas opções: responder com o dado que tem localmente, mesmo sabendo que pode estar desatualizado (escolhendo disponibilidade, abrindo mão de consistência), ou recusar-se a responder até que a partição se resolva e a réplica possa se atualizar (escolhendo consistência, abrindo mão de disponibilidade). Não existe uma terceira opção que preserve as duas garantias ao mesmo tempo enquanto a partição persistir — e é exatamente esse trade-off, inevitável no instante em que a rede falha, que o teorema formaliza.

Um ponto importante, frequentemente mal compreendido, é que partições de rede são inevitáveis em qualquer sistema distribuído real — cabos são cortados, roteadores falham, data centers perdem conectividade entre si. Isso significa que a tolerância a partição (P) não é opcional: qualquer sistema distribuído que se pretenda robusto precisa suportá-la. A escolha CAP que de fato importa na prática, portanto, não é entre C, A e P livremente — é entre C e A, e apenas no instante em que uma partição realmente ocorre. Fora desse instante, que é a maior parte do tempo de operação de um sistema saudável, a grande maioria dos sistemas reais entrega tanto consistência quanto disponibilidade sem problema.

CAP na prática: sistemas CP e sistemas AP

Sistemas CP (Consistência + Tolerância a Partição) escolhem, durante uma partição, recusar ou atrasar respostas em vez de arriscar retornar dados desatualizados. É a escolha natural para dados onde uma resposta errada é pior do que nenhuma resposta — sistemas de coordenação como o Apache ZooKeeper, bancos relacionais configurados com replicação síncrona, e bancos como HBase e MongoDB em sua configuração padrão seguem essa linha, priorizando a integridade do dado sobre a capacidade de sempre responder.

Sistemas AP (Disponibilidade + Tolerância a Partição) escolhem, durante uma partição, continuar respondendo com o dado disponível localmente, mesmo que ele esteja desatualizado, reconciliando as divergências depois que a partição se resolve — tipicamente usando as mesmas estratégias de resolução de conflito discutidas para topologias multi-master. Amazon DynamoDB, Apache Cassandra, Riak e CouchDB são exemplos clássicos dessa escolha, adequada para sistemas onde estar sempre disponível é mais valioso do que estar sempre perfeitamente atualizado — um carrinho de compras que aceita um clique mesmo durante uma instabilidade de rede, por exemplo, geralmente é preferível a um carrinho que trava.

Vale reforçar que essa escolha não é um rótulo fixo e permanente aplicado ao banco de dados inteiro: muitos sistemas modernos permitem configurar o comportamento por operação ou por tabela, e a escolha correta depende do que aquele dado específico representa. Saldo de uma conta bancária tende a exigir CP; contador de visualizações de um vídeo tolera tranquilamente AP. Entender essa distinção — e ser capaz de justificar, para cada dado do seu sistema, se ele exige C ou pode tolerar A durante uma falha de rede — é exatamente a habilidade prática que o teorema CAP existe para desenvolver.

Próxima aula

Replicação e consistência assumem, implicitamente, que sabemos quando uma réplica falhou ou está inacessível — mas como, exatamente, um sistema distribuído percebe que um processo parou de responder, e como distingue um processo lento de um processo morto? A próxima aula abre a unidade de tolerância a falhas propriamente dita: os modelos formais de falha (crash, omissão, bizantina) que os sistemas distribuídos precisam assumir, e os mecanismos práticos de detecção de falhas que sustentam tudo o que vimos até aqui — desde o failover de um master-slave até a exclusão de um nó de um quórum de consenso.

Referências

BREWER, E. A. Towards Robust Distributed Systems (CAP Theorem Keynote). ACM PODC, 2000.

GILBERT, S.; LYNCH, N. Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services. ACM SIGACT News, v. 33, n. 2, 2002.

KLEPPMANN, M. Designing Data-Intensive Applications. Sebastopol: O'Reilly Media, 2017.

TANENBAUM, A. S.; VAN STEEN, M. Sistemas Distribuídos: Princípios e Paradigmas. São Paulo: Pearson.